Module 8: Kernel Machines

Xiao Wang

Department of Statistics
Purdue University

Table of contents

1. Kernels and RKHS

2. Kernel Tricks

3. Kernel Methods for Supervised Learning

Kernels and RKHS

Overview

Motivations:

Develop versatile algorithms to process and analyze data...

...without making any assumptions regarding the type of data (vectors, strings, graphs, images, ...)

The approach:

Develop methods based on pairwise comparisons.

By imposing constraints on the pairwise comparison function (positive definite kernels), we obtain a general framework for learning from data (optimization in RKHS)

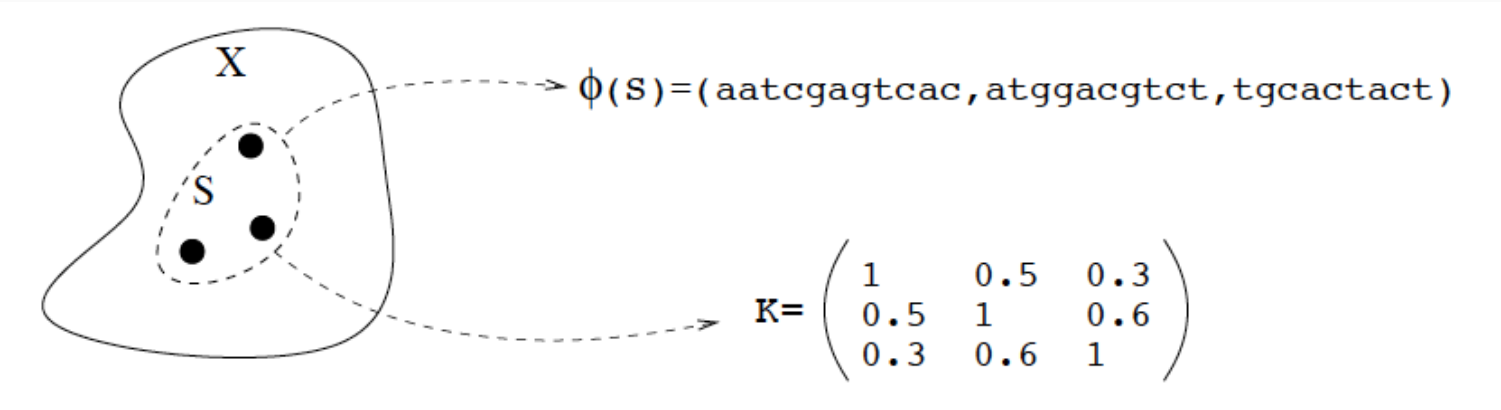
Representation by pairwise comparisons

Idea:

Define a "comparison function": $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$.

Represent a set of n data points $\mathcal{S} = \{x_1, \ldots, x_n\}$ by the $n \times n$ matrix:

$$[K]_{ij} = K(x_i, x_j)$$



- K is always an $n \times n$ matrix, whatever the nature of data: the same algorithm will work for any type of data (vectors, strings, ...).
- Total modularity between the choice of function K and the choice of the algorithm.
- Poor scalability with respect to the dataset size (n^2 to compute and store K)... but how to deal with large-scale problems is possible
- We will restrict ourselves to a particular class of pairwise comparison functions.

Positive definite (p.d.) kernels

Definition:

A positive definite kernel on a set $\mathcal{X}$ is a function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ that is symmetric:

$$\forall\ (x, x') \in \mathcal{X} \times \mathcal{X}, K(x, x') = K(x', x),$$

and which satisfies, for all $N \in \mathbb{N}$, $(x_1, \ldots, x_N) \in \mathcal{X}^N$ and $(a_1, \ldots, a_N) \in \mathbb{R}^N$:

$$\sum_{i=1}^{N} \sum_{j=1}^{N} a_j a_j K(x_i, x_j) \geq 0.$$

- Equivalently, a kernel K is p.d. if and only if, for any $N \in \mathbb{N}$ and any set of points $(x_1, \ldots, x_N) \in \mathcal{X}^N$, the similarity matrix $[K]_{ij} = K(x_i, x_j)$ is positive semidefinite.
- Kernel methods are algorithms that take such matrices as input.

Spectral representation

Define eigenvalues λ_j and orthonormal eigenfunctions ψ_j by

$$\int K(x,y)\psi_j(y)dy = \lambda_j\psi_j(x)$$

$$K(x,y) = \sum_{j=1}^{\infty} \lambda_j\psi_j(x)\psi_j(y)$$

The simplest p.d. kernel for real numbers

Let $\mathcal{X} = \mathbb{R}$. The function $K : \mathbb{R}^2 \to \mathbb{R}$ defined by

$$\forall\ (x, x') \in \mathbb{R}^2, K(x, x') = xx'$$

is p.d.

Proof:

$$xx' = x'x$$

$$\sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j x_i x_j = (\sum_{i=1}^{N} a_i x_i)^2 \geq 0$$

The simplest p.d. kernel for vectors

Let $\mathcal{X} = \mathbb{R}^d$. The function $K : \mathbb{R}^2 \to \mathbb{R}$ defined by

$$\forall\ (x, x') \in \mathbb{R}^d \times \mathbb{R}^d, K(x, x') =< x, x' >$$

is p.d. (it is often called the linear kernel)

Proof:

$$< x, x' >=< x', x >$$

$$\sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j < x_i, x_j >= \| \sum_{i=1}^{N} a_i x_i \|^2 \geq 0$$

A more ambitious p.d. kernel

Let $\mathcal{X}$ be any set, and $\Phi : \mathcal{X} \to \mathbb{R}^d$. Then, the function $K : \mathcal{X}^2 \to \mathbb{R}$ defined as follows is p.d.:

$$\forall\ (x, x') \in \mathcal{X}^2, K(x, x') = \langle \Phi(x), \Phi(x') \rangle$$

Proof:

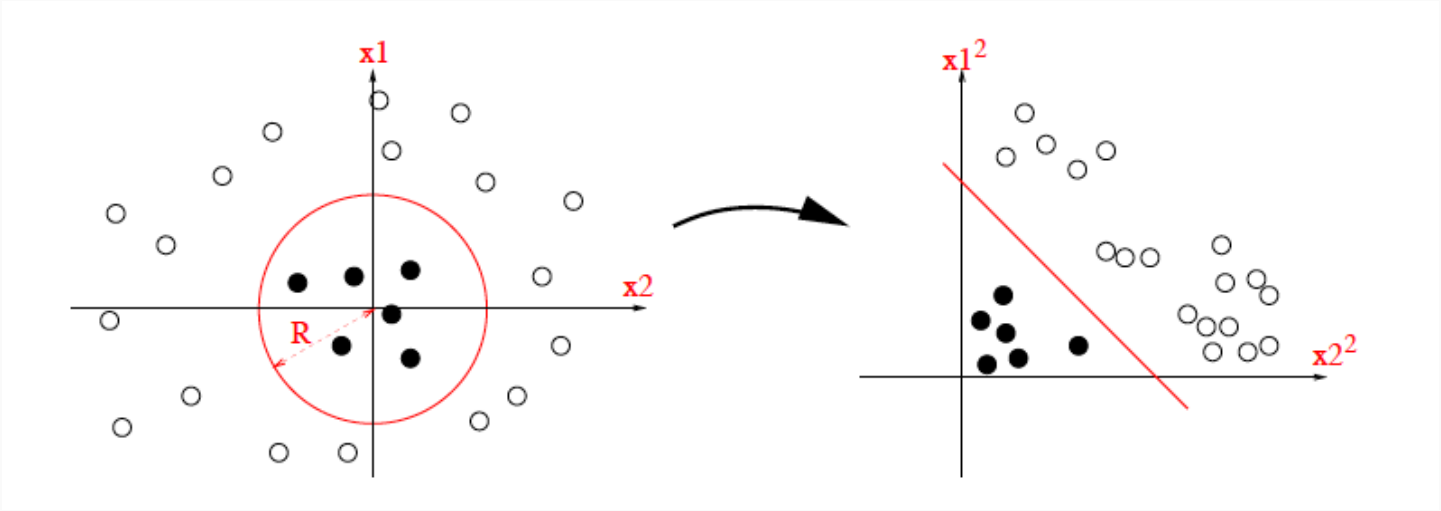
$$< \Phi(x), \Phi(x') >=< \Phi(x'), \Phi(x) >$$

$$\sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j < \Phi(x_i), \Phi(x_j) >= \| \sum_{i=1}^{N} a_i \Phi(x_i) \|^2 \geq 0$$

Example: Polynomial kernel

For $x = (x_1, x_2)^T \in \mathbb{R}^2$, let $\Phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^3$:

$$\begin{aligned} K(x, x') &= x_1^2 {x_1'}^2 + 2x_1x_2x_1'x_2' + x_2^2 {x_2'}^2 \\ &= (x_1x_1' + x_2x_2')^2 \\ &= \langle x, x' \rangle^2 \end{aligned}$$



Exercise: Show that $\langle x, x' \rangle^d$ is p.d. on $\mathcal{X} = \mathbb{R}^p$ for an $d \in \mathbb{N}$.

Conversely: Kernels as inner product

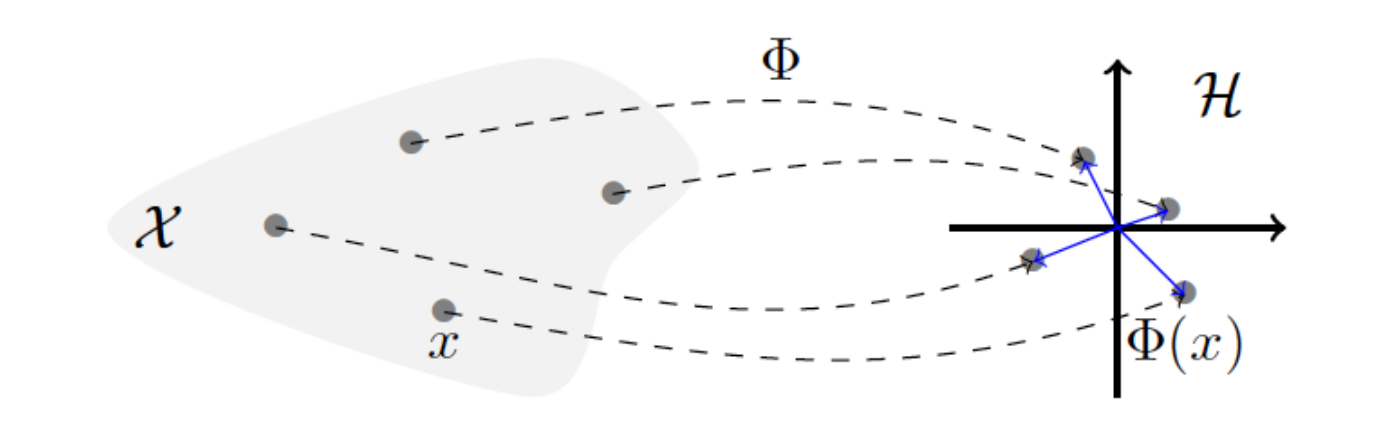
Theorem (Aronszajn, 1950):

K is a p.d. kernel on the set $\mathcal{X}$ if and only if there exists a Hilbert space $\mathcal{H}$ and a mapping

$$\Phi : \mathcal{X} \to \mathcal{H}$$

such that, for any $x, x' \in \mathcal{X}$:

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}}.$$



Functional spaces for machine learning

Before we go into formal details:

- Among the Hilbert spaces $\mathcal{H}$ mentioned in Aronszjan's theorem, we will see that one of them, called RKHS, is of interest to us.
- This is a space of functions from $\mathcal{X}$ to $\mathbb{R}$.
- In other words, each data point x in $\mathcal{X}$ will be represented by a function $\Phi(x) = K_x$ in $\mathcal{H}$.

Example of functional mapping:

- Consider $\mathcal{X} = \mathbb{R}$. We could decide to represent each scalar $x \in \mathbb{R}$ as a Gaissian function centered at x:

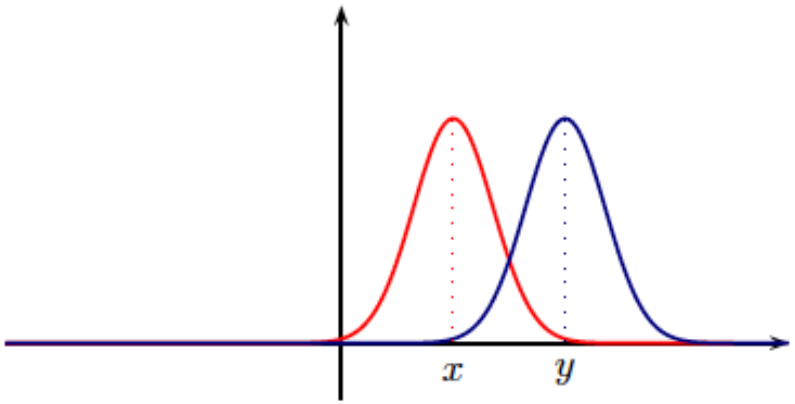
$$K_x : y \to e^{-\frac{1}{2\alpha}(x-y)^2}$$

- What could be the corresponding $\mathcal{H}$ (if it exists)? What would be the inner-product?

What does it mean to map a data point to a function?

$$\Phi(x) : t \mapsto e^{-\frac{1}{2\alpha^2}(x-t)^2}$$

$$\Phi(y) : t \mapsto e^{-\frac{1}{2\alpha^2}(y-t)^2}$$



Data points are mapped to Gaussian functions living in a Hilbert space $\mathcal{H}$

But $\mathcal{H}$ is much richer and contains much more than Gaussian functions!

RKHS definition

Let $\mathcal{X}$ be a set and $\mathcal{H}$ be a class of functions forming a real Hilbert space with inner product $<,>_{\mathcal{H}}$. The function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a reproduction kernel (r.k.) of $\mathcal{H}$ if

1. $\mathcal{H}$ contains all functions of the form

$$\forall\, x \in \mathcal{X}, K_x : t \to K(x, t).$$

2. For every $x \in \mathcal{X}$ and $f \in \mathcal{H}$, the reproducing property holds:

$$f(x) = \langle f, K_x \rangle_{\mathcal{H}}.$$

If a r.k. exists, then $\mathcal{H}$ is called a reproducing kernel Hilbert space (RKHS).

Recall $K(x, y) = \sum_{j=1}^{\infty} \lambda_j \psi_j(x) \psi_j(y)$.

Define the feature map

$$\Phi(x) = (\sqrt{\lambda_1}\psi_1(x), \sqrt{\lambda_2}\psi_2(x), \cdots)$$

We can expand f in terms of the basis $\psi_1, \psi_2, \ldots$:

$$f(x) = \sum_{j=1}^{\infty} \beta_j \psi_j(x).$$

If $f(x) = \sum_{j=1}^{\infty} a_j \psi_j(x)$ and $g(x) = \sum_{j=1}^{\infty} b_j \psi_j(x)$

$$\langle f, g \rangle_{\mathcal{H}} = \sum_{j=1}^{\infty} \frac{a_j b_j}{\lambda_j}$$

Roughly speaking, when $\|f\|_{\mathcal{H}}$ is small, then f is smooth.

RKHS: Why do we care?

- Map data x in $\mathcal{X}$ to a high-dimensional Hilbert space $\mathcal{H}$ (the RKHS) through a kernel mapping $\Phi : \mathcal{X} \to \mathcal{H}$, with $\Phi(x) = K_x$.
- In $\mathcal{H}$, consider simple linear models $f(x) = \langle f, \Phi(x) \rangle_{\mathcal{H}}$.
- If $X = \mathbb{R}^p$, a linear function in $\Phi(x)$ may be nonlinear in x.
- For instance, for supervised learning, given training data $(y_i, x_i)_{i=1}^n$, we may want to minimize the empirical risk

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$$

  More formal details to come ...

Kernel Tricks

Motivations

Two theoretical results underpin a family of powerful algorithms for data analysis using p.d. kernels, collectively known as kernel methods:

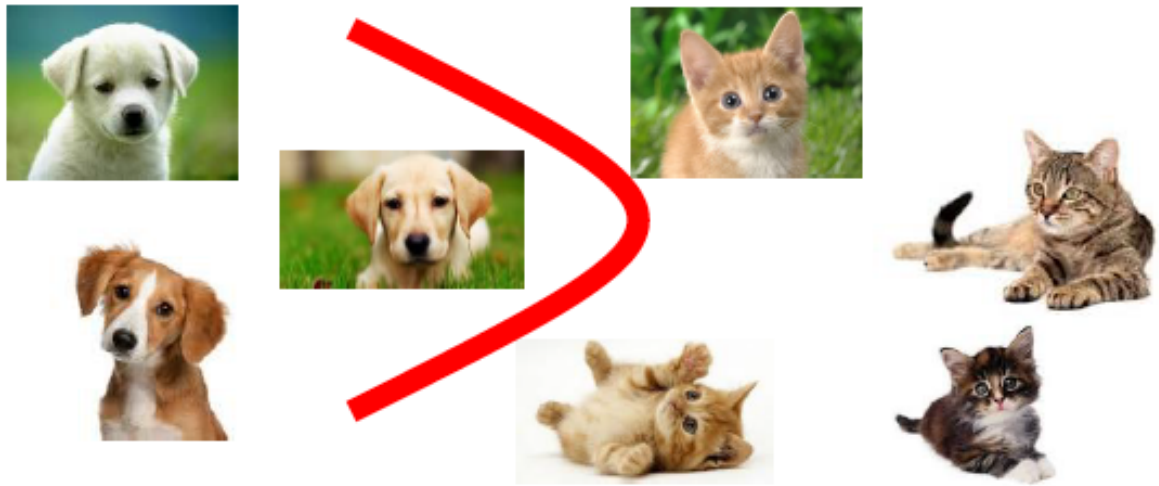
— The kernel trick, based on the representation of p.d. kernels as inner products;

— The representer theorem, based on some properties of the regularization functional defined by the RKHS norm.

Motivation from supervise learning

For instance, in supervised learning, the goal is to learn a prediction function $f : \mathcal{X} \to \mathcal{Y}$ given labeled training data $(x_i, y_i)_{i=1}^n$ with $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$:

$$\min_{f \in \mathcal{F}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i))}_{\text{empirical risk, data fit}} + \underbrace{\lambda \Omega(f)}_{\text{regularization}} .$$

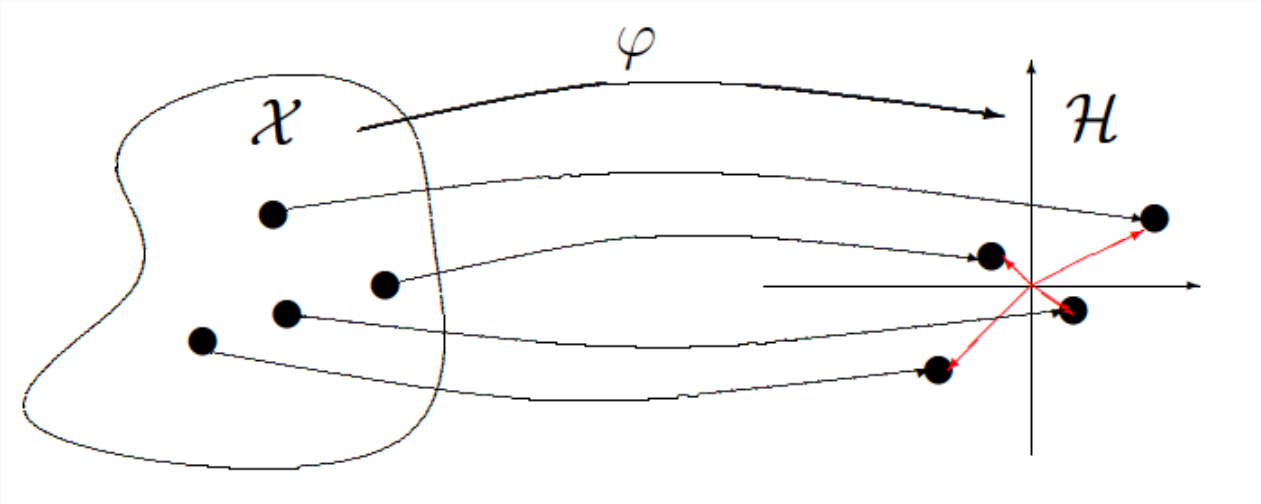


Example with linear model: logistic regression, etc

- Assume there exists a linear relation between y and features x in $\mathbb{R}^p$
- $f(x) = w^T x + b$ is parameterized by $w, b \in \mathbb{R}^{p+1}$
- L is often a convex loss function
- $\Omega(f)$ is often the squared ℓ_2-norm $\|w\|_2^2$

Kernel methods allow you to map data x in $\mathcal{X}$ to a Hilbert space and work with linear forms

$$\Phi : \mathcal{X} \to \mathcal{H} \quad \text{and} \quad f(x) = \langle \Phi(x), f \rangle_{\mathcal{H}}$$



First purpose: embed data in a vectorial space where:

— many geometrical operations exist (angle computation, projection on linear subspaces, definition of barycenters....).

— one may learn potentially rich infinite-dimensional models.

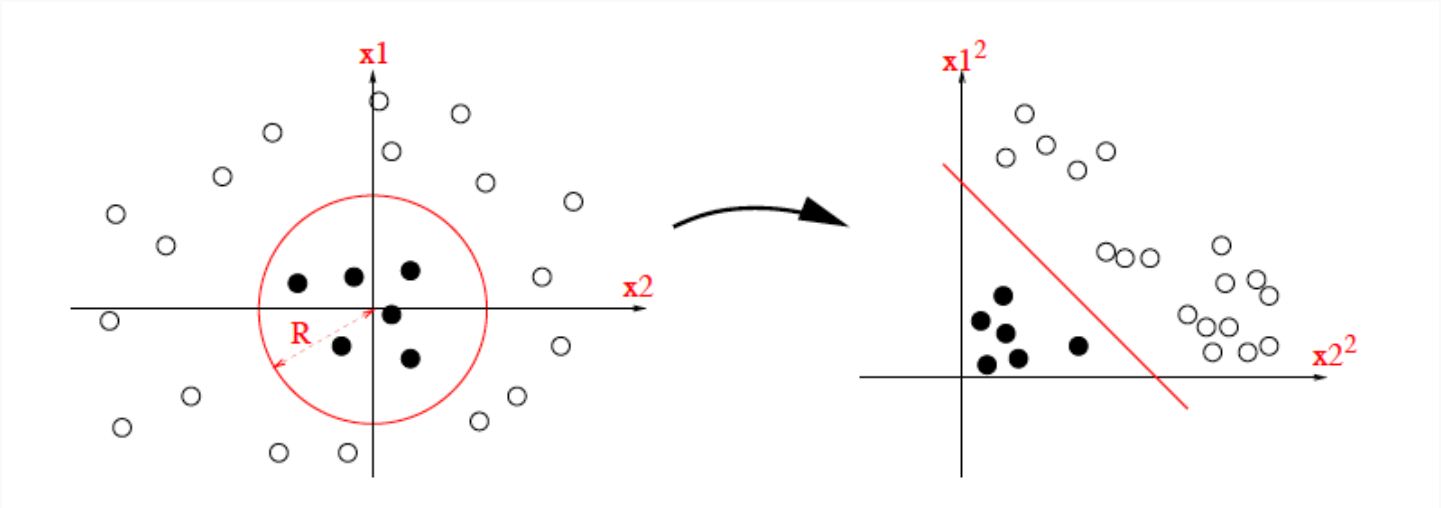
— regularization is natural and theoretically grounded.

The principle is generic and does not assume anything about the nature of the set $\mathcal{X}$ (vectors, sets, graphs, sequences).

Second purpose: unhappy with the current Euclidean structure?

— lift data to a higher-dimensional space with nicer properties (e.g., linear separability, clustering structure).

— then, the linear form $f(x) = \langle \Phi(x), f \rangle_{\mathcal{H}}$ in $\mathcal{H}$ may correspond to a non-linear model in $\mathcal{X}$.



The kernel trick

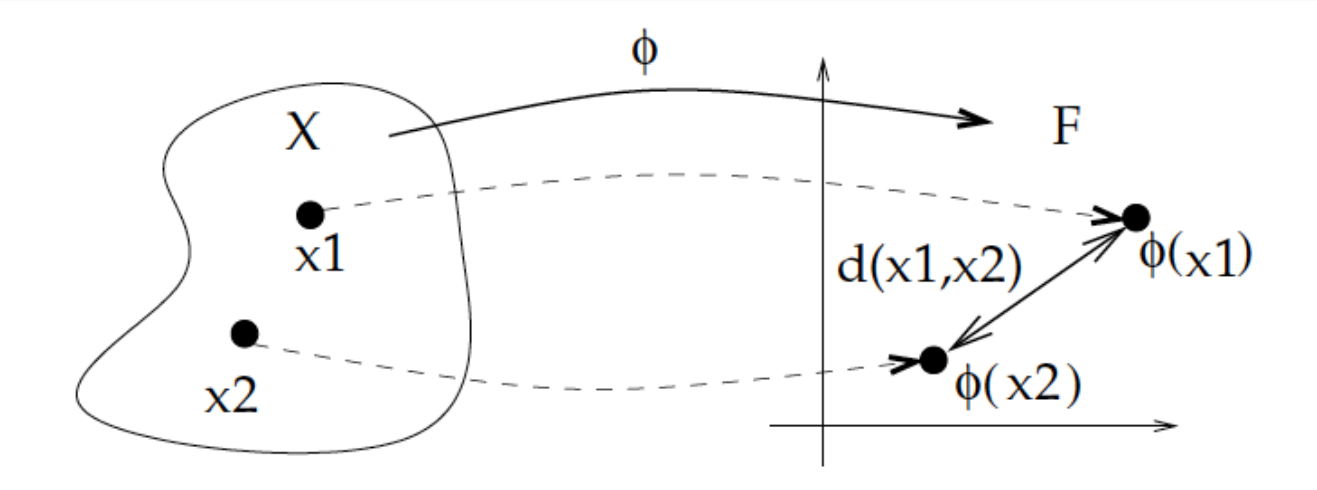
Proposition:

Any algorithm to process finite-dimensional vectors that can be expressed only in terms of pairwise inner products can be applied to potentially infinite-dimensional vectors in the feature space of a p.d. kernel by replacing each inner product evaluation by a kernel evaluation.

Remarks:

- The proof of this proposition is trivial, because the kernel is exactly the inner product in the feature space.
- This trick has huge practical applications.
- Vectors in the feature space are only manipulated implicitly, through pairwise inner products.

Example: computing distances in the feature space



$$
\begin{aligned}
d_K(x_1, x_2)^2 &= \|\Phi(x_1) - \Phi(x_2)\|^2_{\mathcal{H}} \\
&= \langle \Phi(x_1), \Phi(x_1) \rangle_{\mathcal{H}} + \langle \Phi(x_2), \Phi(x_2) \rangle_{\mathcal{H}} - 2\langle \Phi(x_1), \Phi(x_2) \rangle_{\mathcal{H}} \\
&= K(x_1, x_1) + K(x_2, x_2) - 2K(x_1, x_2)
\end{aligned}
$$

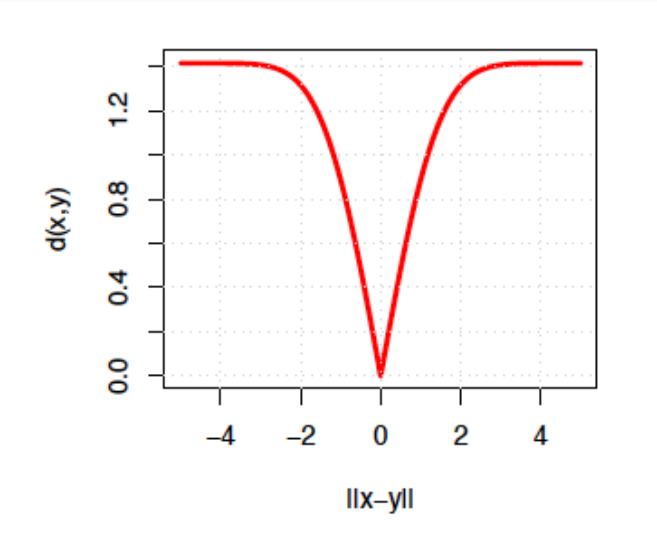
Distance for the Gaussian kernel

- The Gaussian kernel with bandwidth σ on $\mathbb{R}^d$ is

$$K(x, y) = e^{-\frac{\|x-y\|^2}{2\sigma^2}}$$

- $K(x, x) = 1 = \|\Phi(x)\|^2_{\mathcal{H}}$, so all points are on the unit sphere in the feature space.
- The distance between the images of two points x and y in the feature space is given by

$$d_K(x, y) = \sqrt{2[1 - e^{-\frac{\|x-y\|^2}{2\sigma^2}}]}$$



Kernel trick summary

- The kernel trick is a trivial statement with important applications.
- It can be used to obtain nonlinear versions of well-known linear algorithms, e.g., by replacing the classical inner product by a Gaussian kernel.
- It can be used to apply classical algorithms to non vectorial data (e.g., strings, graphs) by again replacing the classical inner product by a valid kernel for the data.
- It allows in some cases to embed the initial space to a larger feature space and involve points in the feature space with no pre-image (e.g., barycenter).

Representer theorem: motivation

- An RKHS is a space of (potentially nonlinear) functions, and $\|f\|_{\mathcal{H}}$ measures the smoothness of f.
- Given a set of data $(x_i \in \mathcal{X}, y_i \in \mathbb{R})_{i=1}^n$, a natural way to estimate a regression function $f : \mathcal{X} \to \mathbb{R}$ is to solve something like:

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$$

- How to solve in practice this problem, potentially in infinite dimension?

The representer theorem

- Let $\mathcal{X}$ be a set endowed with a p.d. kernel K, $\mathcal{H}$ the corresponding RKHS, and $\mathcal{S} = \{x_1, \ldots, x_n\} \subseteq \mathcal{X}$ a finite set of points in $\mathcal{X}$.
- Let $\Psi : \mathbb{R}^{n+1} \to \mathbb{R}$ be a function of $n+1$ variables, strictly increasing with respect to the last variable.
- Then, any solution to the optimization problem

$$\min_{f \in \mathcal{H}} \Psi(f(x_1), \cdots, f(x_n), \|f\|_{\mathcal{H}}),$$

admits a representation of the form

$$\forall x \in \mathcal{X}, f(x) = \sum_{i=1}^{n} a_i K(x_i, x) = \sum_{i=1}^{n} a_i K_{x_i}(x).$$

In other words, the solution lives in a finite-dimensional subspace:

$$f \in \mathrm{Span}(K_{x_1}, \ldots, K_{x_n}).$$

Remarks

Often the function Ψ has the form:

$$\Psi(f(x_1), \cdots, f(x_n), \|f\|_{\mathcal{H}}) = c(f(x_1), \ldots, f(x_n)) + \lambda\Omega(\|f\|_{\mathcal{H}})$$

Theoretically, the minimization will enforce the norm $\|f\|_{\mathcal{H}}$ to be small, which can be beneficial by ensuring a sufficient level of smoothness for the solution (regularization effect).

Practically, we know by the representer theorem that the solution lives in a subspace of dimension n, which can lead to efficient algorithms although the RKHS itself can be of infinite dimension.

Kernel Methods for Supervised Learning

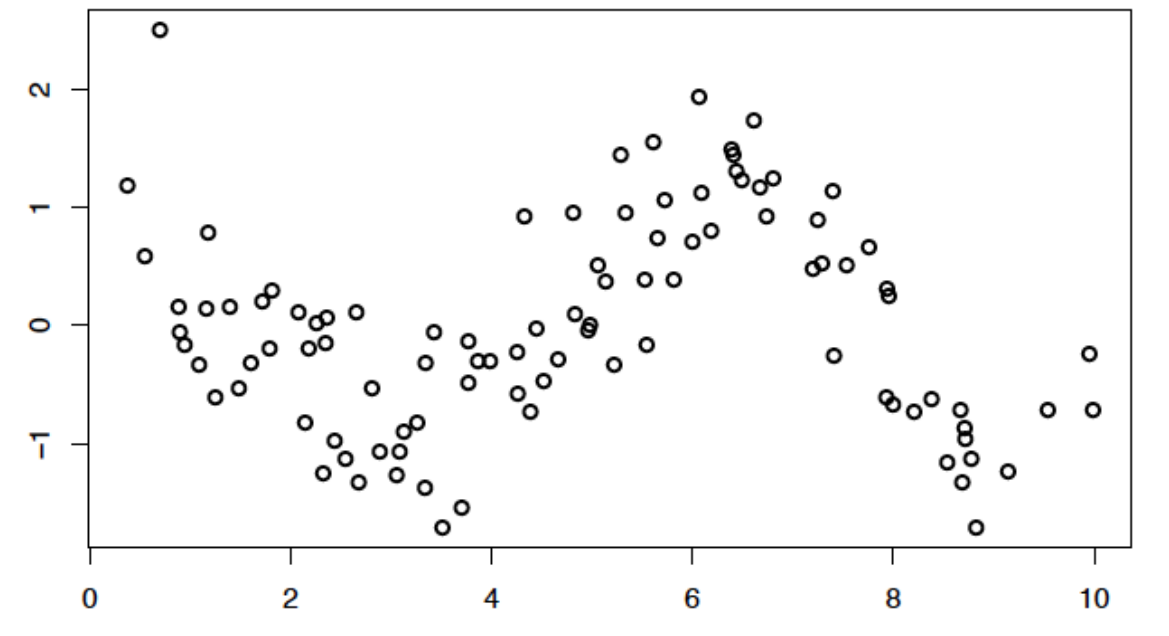
General setting

$$\min_{f \in \mathcal{H}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \ell\left(f(\mathbf{x}_i), y_i\right)}_{\text{empirical risk, data fit}} + \underbrace{\lambda \| f \|_{\mathcal{H}}^2}_{\text{regularization}} .$$

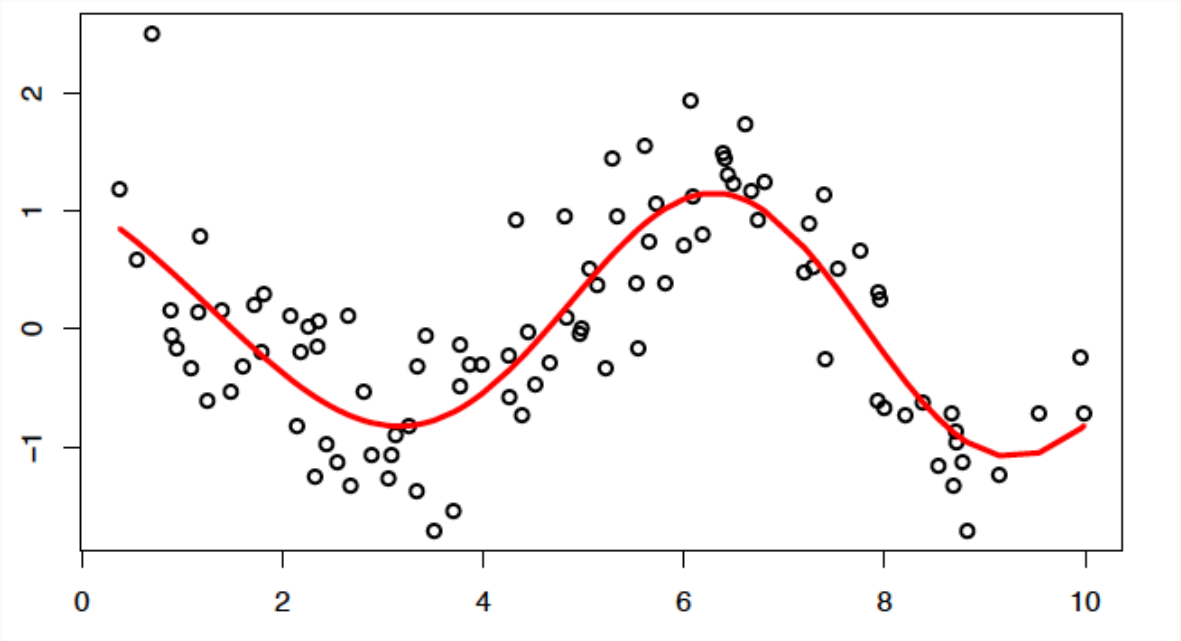
- Regularization is important, particularly in high dimension, to prevent overfitting
- When $\mathcal{X} = \mathbb{R}^d$ and K is a linear kernel, $f = f_w$ is a linear model and the regularization is $\|w\|^2$
- Using more general spaces and kernels allows to
  — learn non-linear functions over a functional space endowed with a natural regularization (remember, small norm in RKHS = "smooth")
  — learn functions over non-vectorial data, such as strings and graphs

We will now see a few methods in more details.

Regression



Goal: Find a $f : \mathcal{X} \to \mathbb{R}$ to predict y by $f(x)$


2
1
0
−1
0
2
4
6
8
10

Least square regression over a general functional space

- Let us quantify the error if f predicts $f(x)$ instead of y by the squared error

$$\ell(f(x), y) = (y - f(x))^2$$

- Fix a set of functions $\mathcal{H}$
- Least square regression amounts to find the function in $\mathcal{H}$ with the smallest empirical risk, call in this case the mean squared error (MSE):

$$\hat{f} \in \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} (y_i - f(x_i))^2$$

- Issues: unstable (especially in large dimensions), overfitting if $\mathcal{H}$ is too large.

Kernel ridge regression (KRR)

- Let us now consider a RKHS $\mathcal{H}$, associated to a p.d. kernel K on $\mathcal{X} \times \mathcal{X}$.
- KRR is obtained by regularizing the MSE criterion by the RKHS norm:

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2$$

- 1st effect = prevent overfitting by penalizing non-smooth functions
- By the representer theorem, any solution can be expanded as

$$\hat{f}(x) = \sum_{i=1}^{n} a_i K(x_i, x)$$

- 2nd effect = simplifying the solution

Solving KRR

- Let $y = (y_1, \ldots, y_n) \in \mathbb{R}^n$
- Let $a = (a_1, \ldots, a_n) \in \mathbb{R}^n$
- Let K be the $n \times n$ Gram matrix: $K_{ij} = K(x_i, x_j)$
- We can write

$$(\hat{f}(x_1), \ldots, \hat{f}(x_n))^T = Ka$$

- The penalty term:

$$\|\hat{f}\|_{\mathcal{H}}^2 = a^T Ha$$

- The KRR problem is equivalent to

$$\arg\min_{a \in \mathbb{R}^n} \frac{1}{n}(Ka - y)^T(Ka - y) + \lambda a^T Ka$$

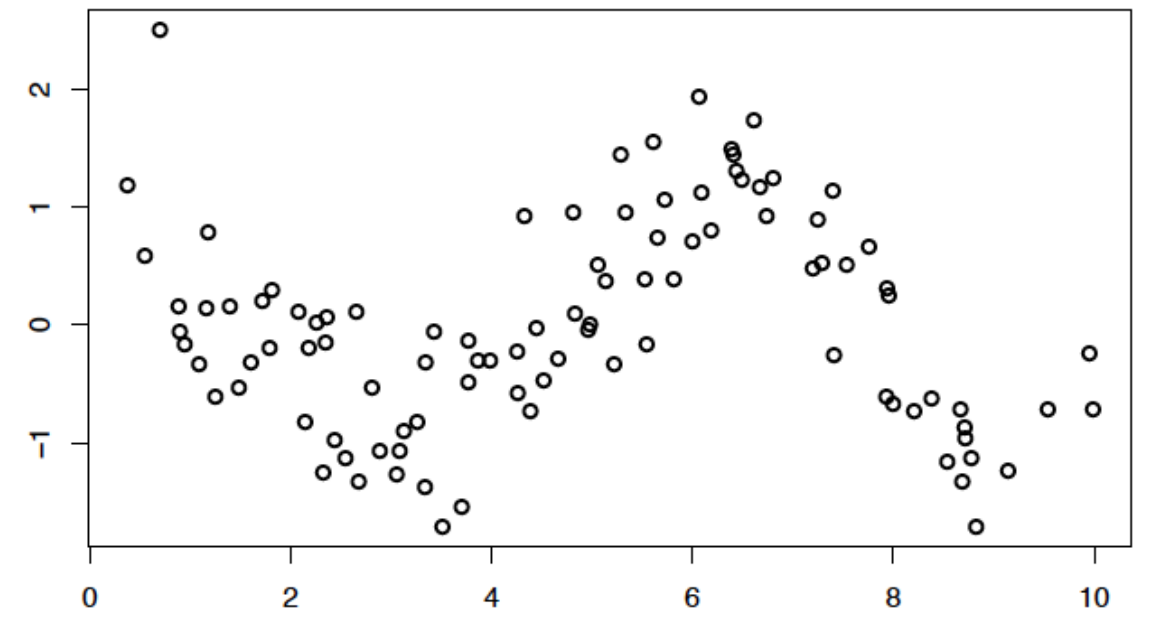
- This is a convex and differentiable function of a. Its minimum can therefore be found by setting the gradient in a to zero:

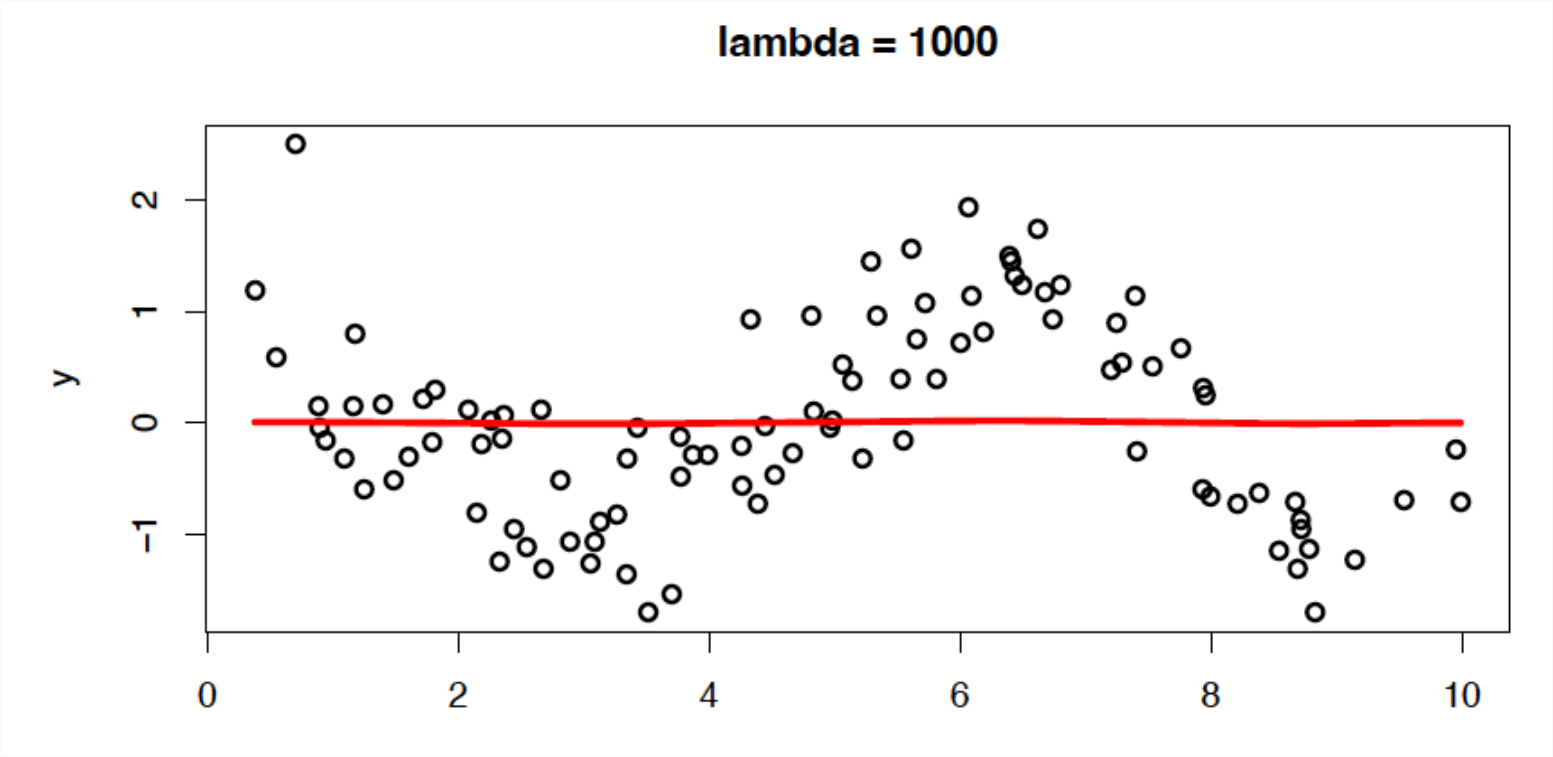
$$0 = \frac{2}{n} K(Ka - y) + 2\lambda Ka = K[(K + \lambda n I)a - y]$$

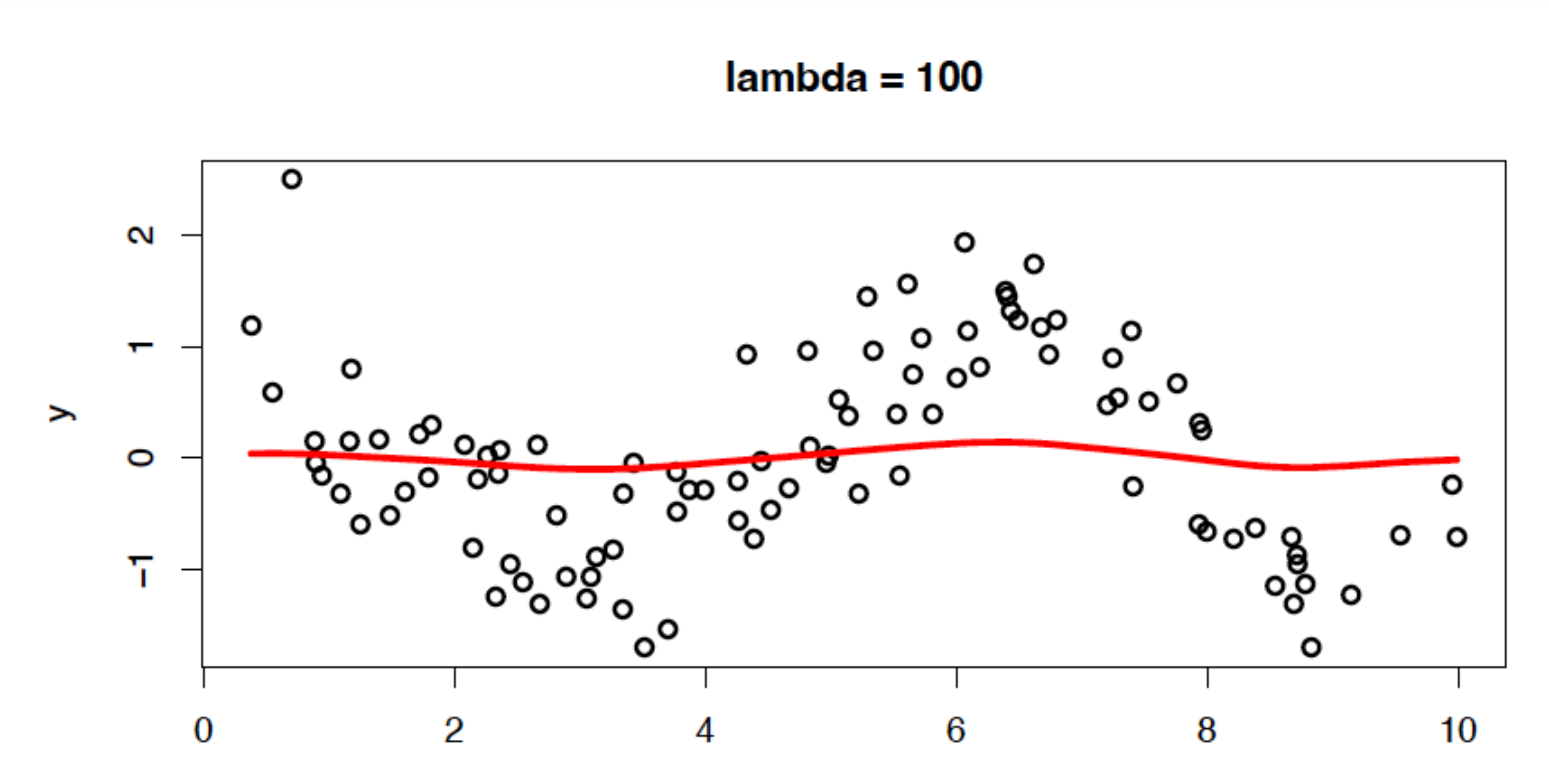
- For $\lambda > 0$, $K + \lambda n I$ in invertible (because K is positive semidefinite), so one solution is to take

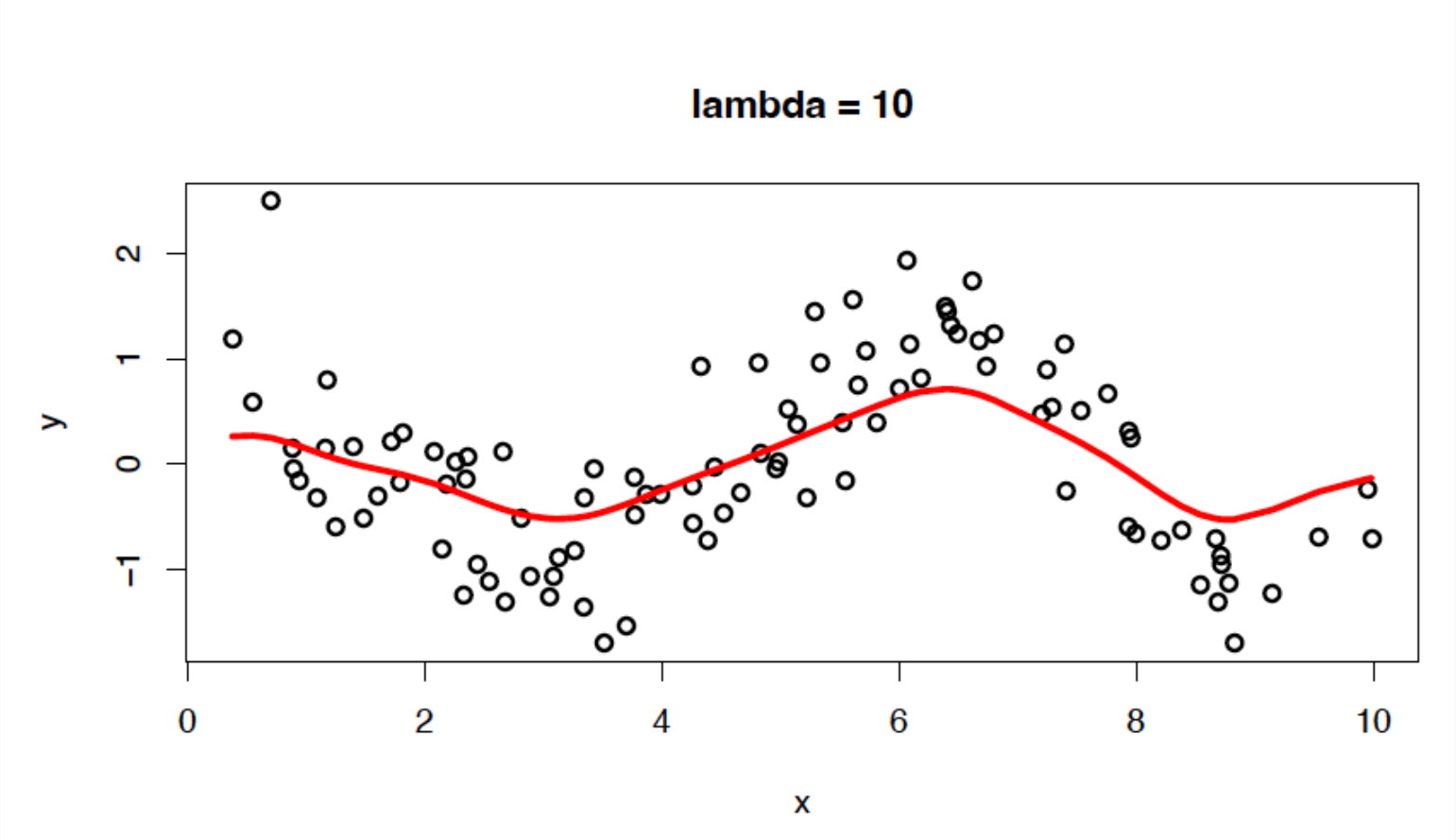
$$a = (K + \lambda n I)^{-1} y$$

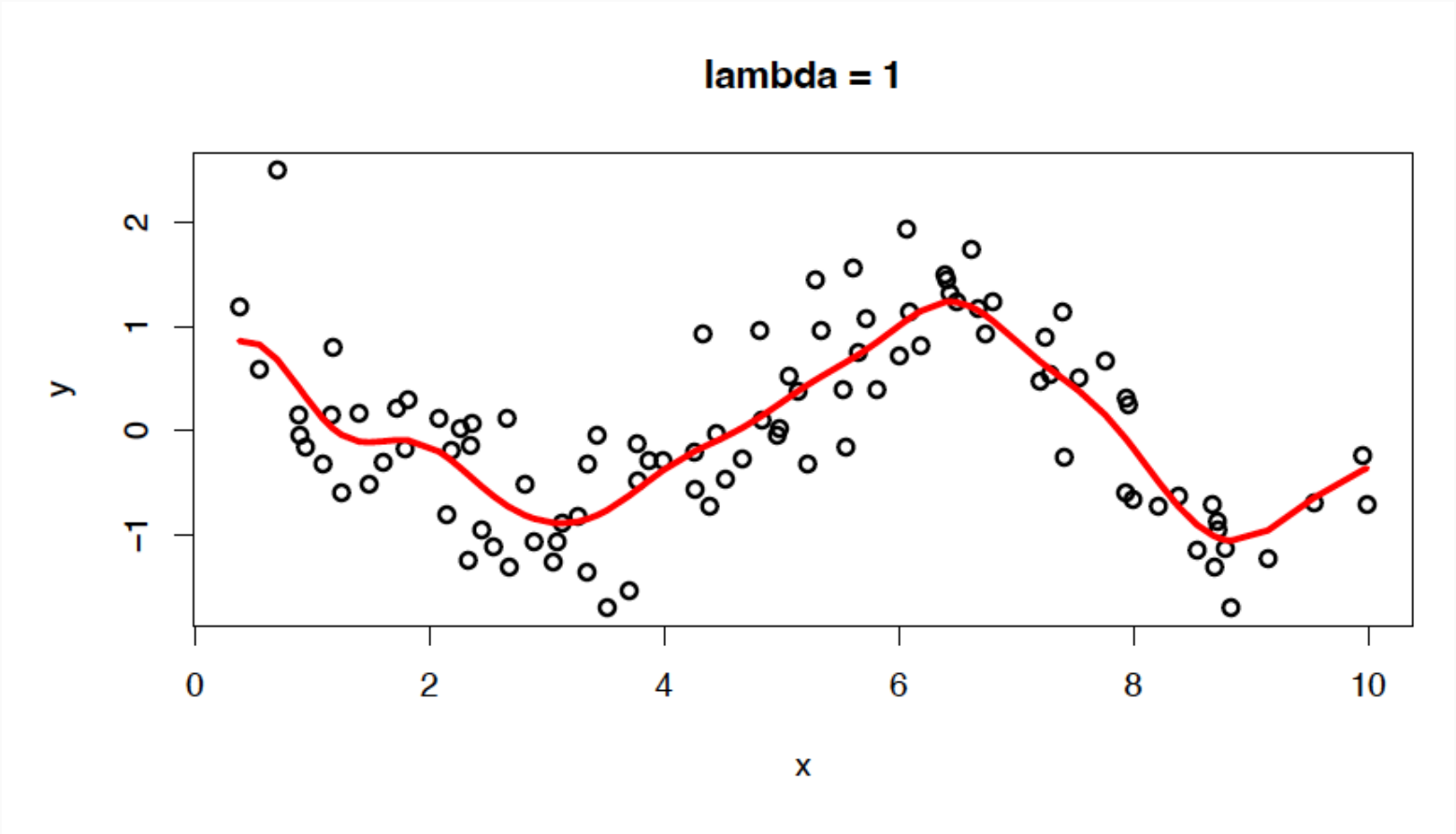
Example: KRR with Gaussian RBF kernel

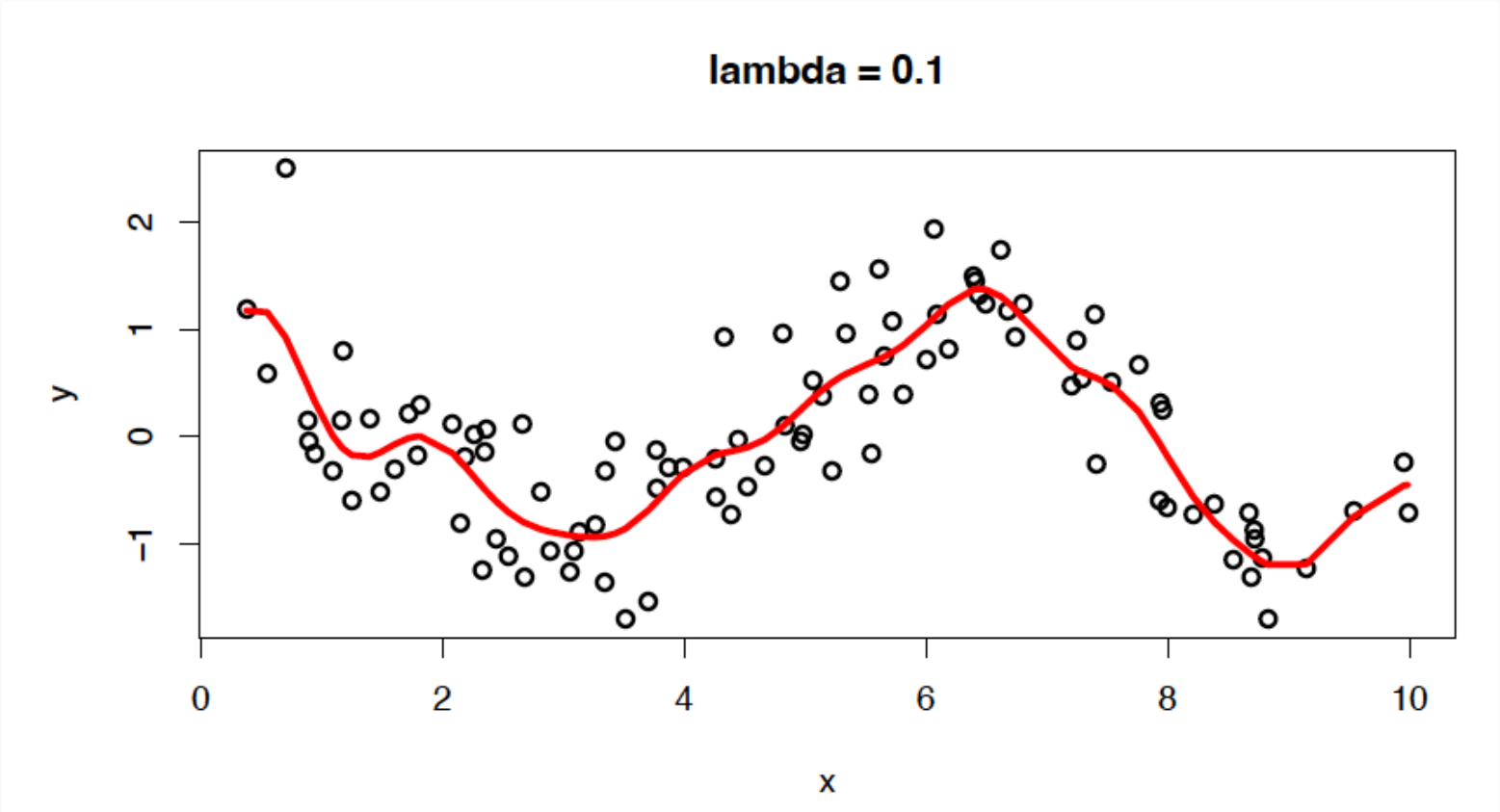


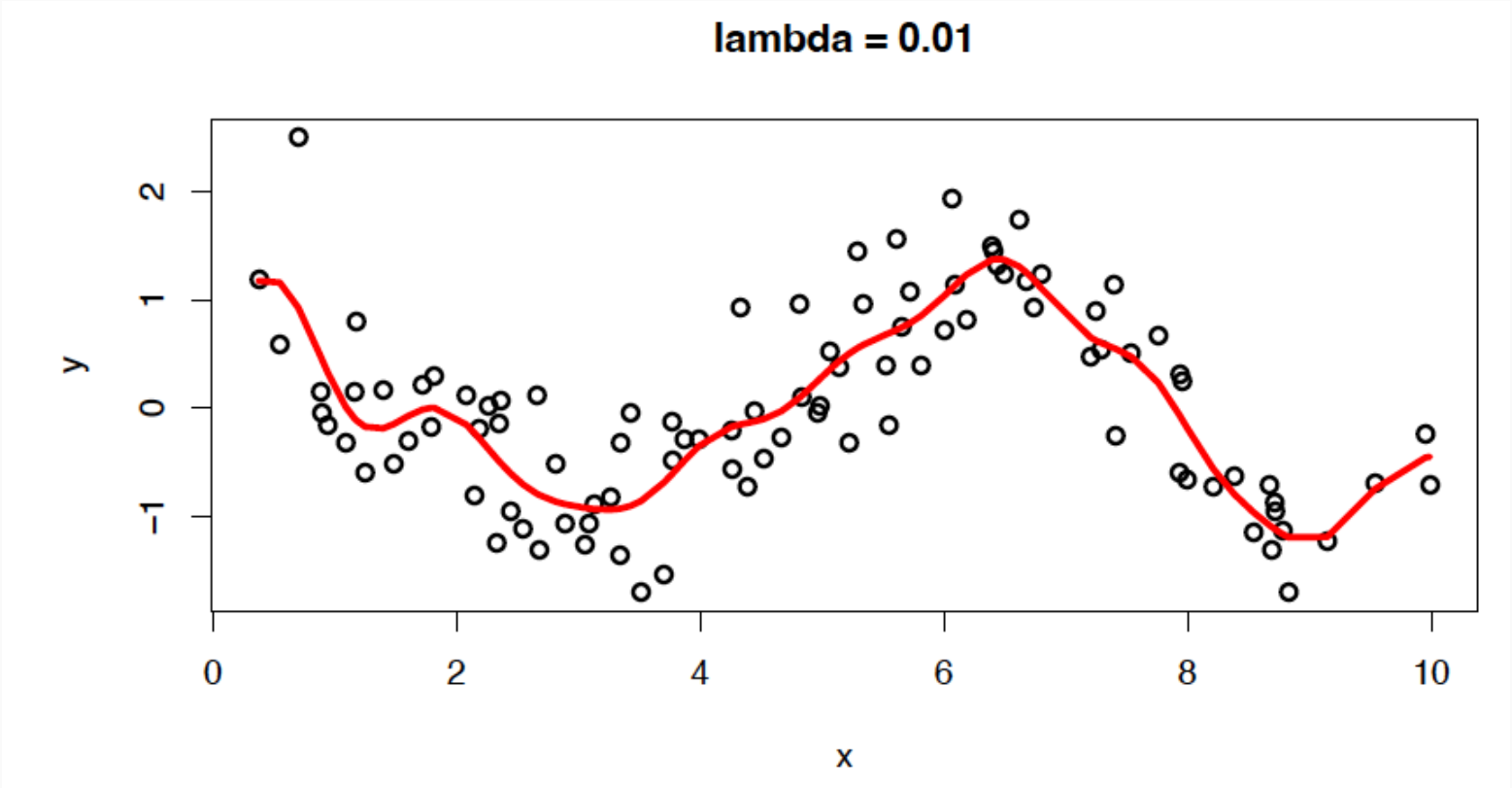

lambda = 1000
y
2
1
0
−1
0
2
4
6
8
10

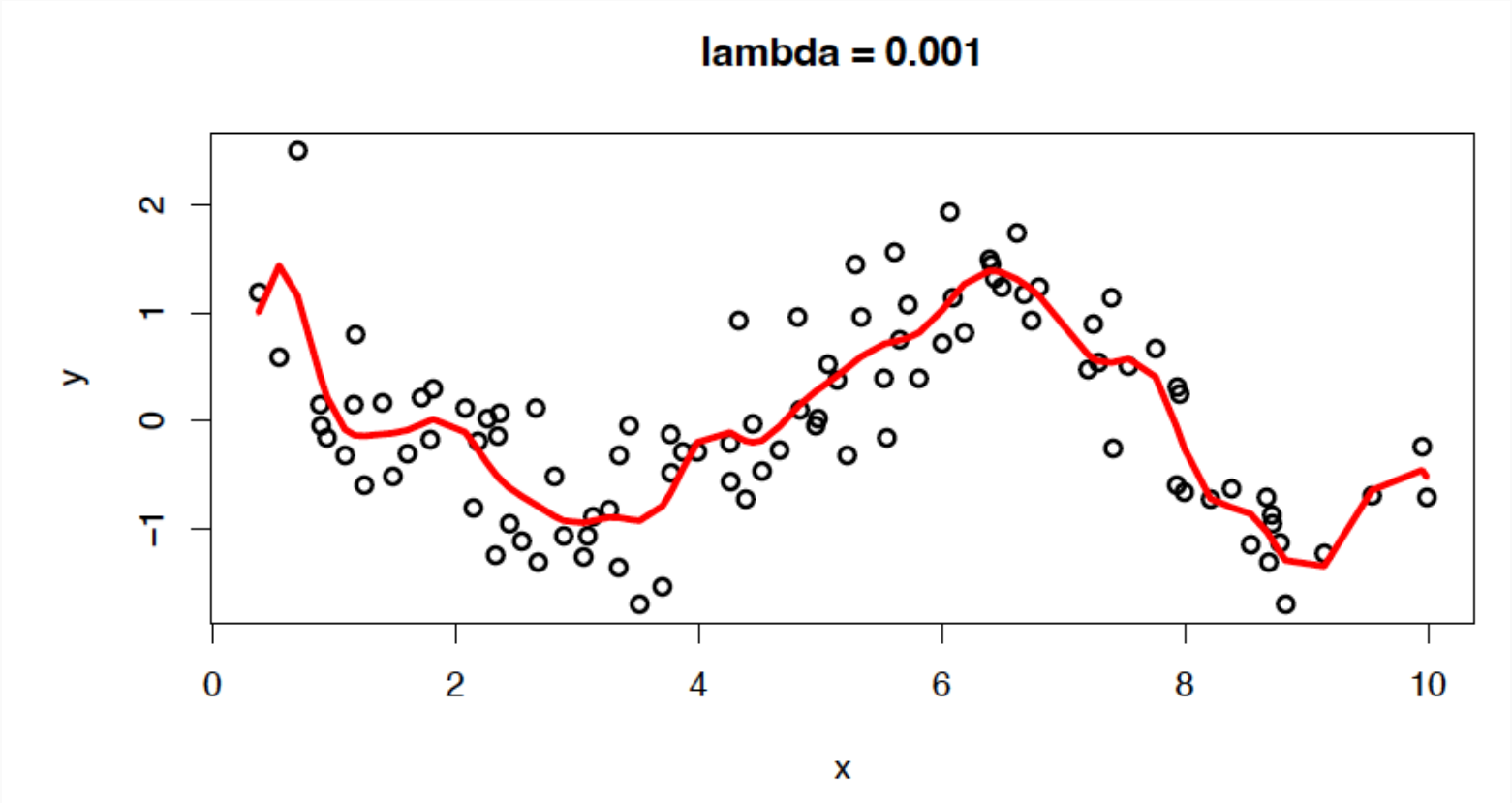

lambda = 100
y
2
1
0
−1
0
2
4
6
8
10

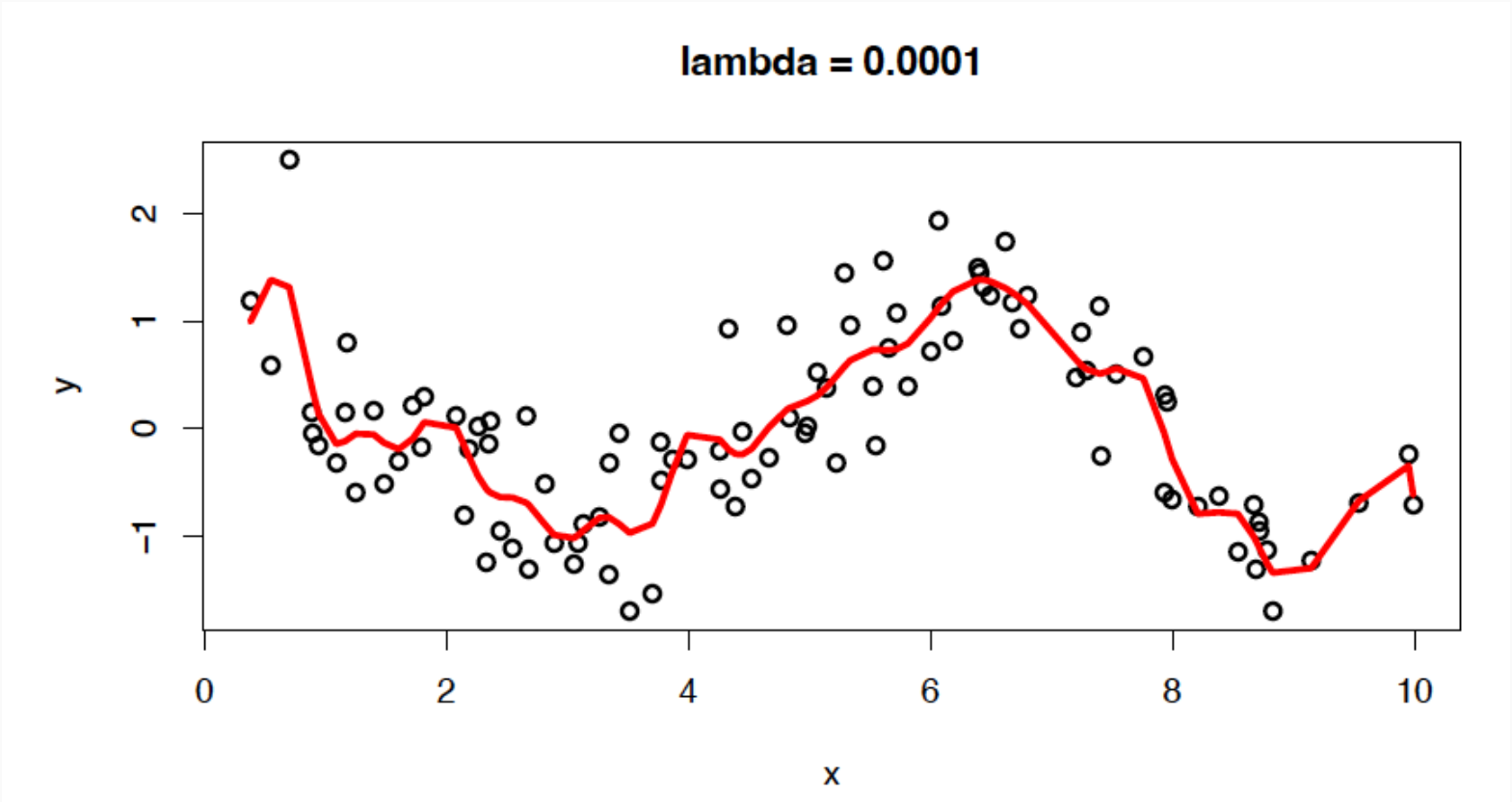

lambda = 10
y
x
−1
0
1
2
0
2
4
6
8
10

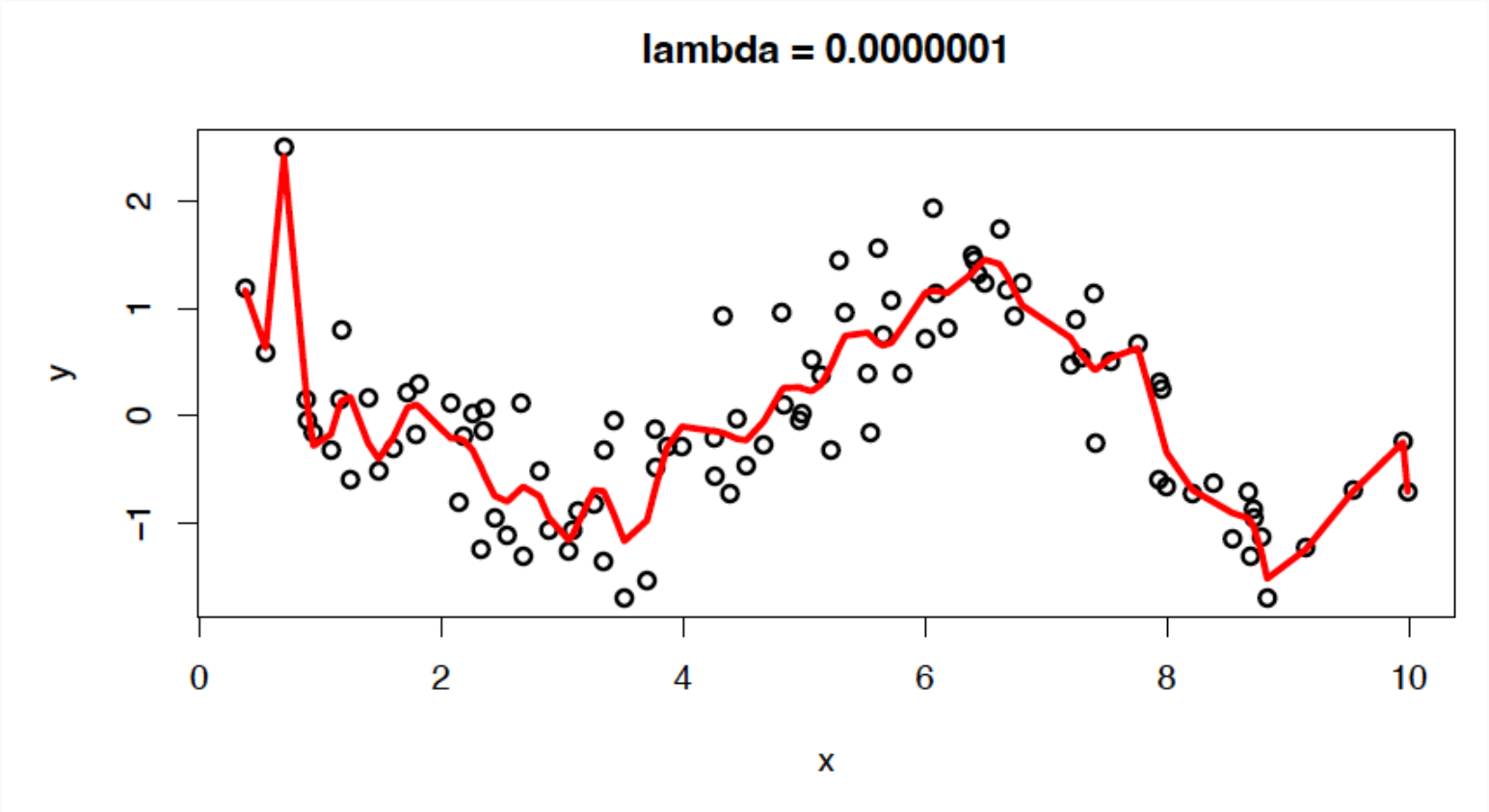

lambda = 1
y
x
−1
0
1
2
0
2
4
6
8
10


lambda = 0.1
y
x
−1
0
1
2
0
2
4
6
8
10


lambda = 0.01
y
x
2
1
0
−1
0
2
4
6
8
10


lambda = 0.001
y
2
1
0
−1
0
2
4
6
8
10
x


lambda = 0.0001
y
2
1
0
−1
0
2
4
6
8
10
x


lambda = 0.0000001
y
x
−1
0
1
2
0
2
4
6
8
10

Weighted regression

- Given weights $W_1, \ldots, W_n \in \mathbb{R}$, a variant of ridge regression to weight differently the error at different points:

$$\arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} W_i (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2$$

- By the representer theorem the solution is $f(x) = \sum_{i=1}^{n} a_i K(x_i, x)$, where a solves, with $W = \mathrm{diag}(W_1, \ldots, W_n)$:

$$\arg\min_{a \in \mathbb{R}^n} \frac{1}{n} (Ka - y)^T W (Ka - y) + \lambda a^T K a$$

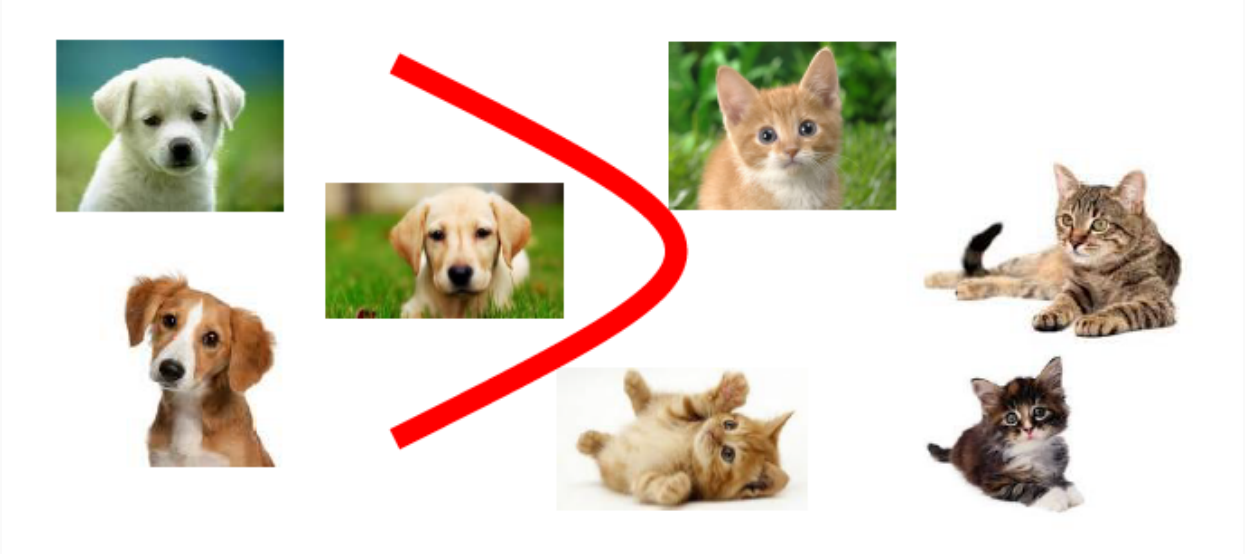
- A solution is

$$a = W^{1/2} (W^{1/2} K W^{1/2} + n\lambda I)^{-1} W^{1/2} y$$

Binary classification



Goal = find a function $f : \mathcal{X} \to \mathbb{R}$ to predict y by $\text{sign}(f(x))$



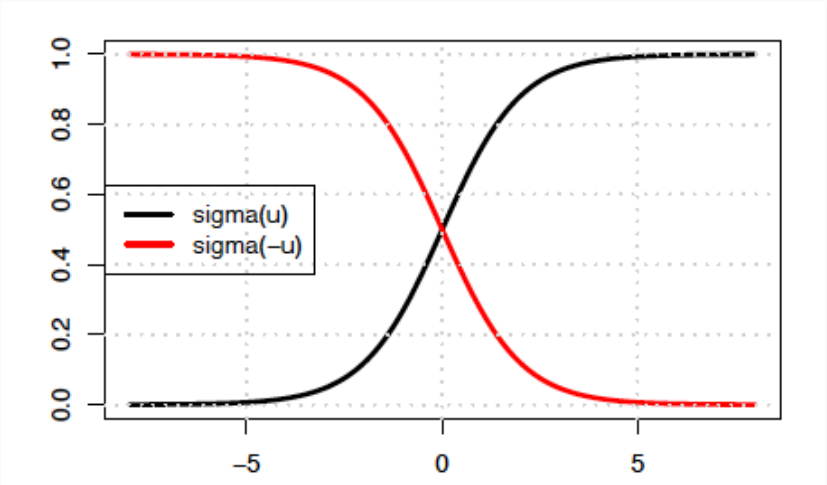
The 0-1 loss

- The 0-1 loss is
$$\ell_{0-1}(f(x), y) = 1(yf(x) < 0)$$
- It is them tempting to learn f by solving
$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell_{0-1}(f(x_i), y_i) + \lambda \|f\|_{\mathcal{H}}^2$$
- However:
  — The problem is non-smooth, and typically NP-hard to solve
  — The regularization has no effect since the 0-1 loss is invariant by scaling of f

The logistic loss

- An alternative is to define a probabilistic model of y parametrized by $f(x)$:

$$\forall\ y \in \{+1, -1\}, P(y|x) = \frac{1}{e^{-yf(x)}} = \sigma(yf(x))$$



- The logistic loss is the negative conditional likelihood

$$\ell_{logistic}(f(x), y) = -\log p(y|x) = \log(1 + e^{-yf(x)})$$

Kernel logistic regression (KLR)

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell_{logistic}(f(x_i), y_i) + \frac{\lambda}{2} \|f\|_{\mathcal{H}}^2$$

$$= \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \log(1 + e^{-y_i f(x_i)}) + \frac{\lambda}{2} \|f\|_{\mathcal{H}}^2$$

Can be interpreted as a regularized conditional maximum likelihood estimator

No explicit solution, but smooth convex optimization problem that can be solved numerically

Solving KLR

- By the representer theorem, any solution of KLR can be expanded as

$$\hat{f}(x) = \sum_{i=1}^{n} a_i K(x_i, x)$$

and as always we have

$$(\hat{f}(x_1), \ldots, \hat{f}(x_n))^T = Ka, \quad \|\hat{f}\|_{\mathcal{H}}^2 = a^T Ka$$

- To find a we need to solve

$$\min_{a \in \mathbb{R}^n} J(a) = \frac{1}{n} \sum_{i=1}^{n} \log(1 + e^{-y_i [Ka]_i}) + \frac{\lambda}{2} a^T Ka$$

This is a smooth convex optimization problem, that can be solved by many numerical methods. Let us explicit one of them, Newton's method, which iteratively approximates J by a quadratic function and solves the quadratic problem.

Similar to linear logistic regression, it can be changed to iteratively solve a standard weighted kernel ridge regression (WKRR) problem

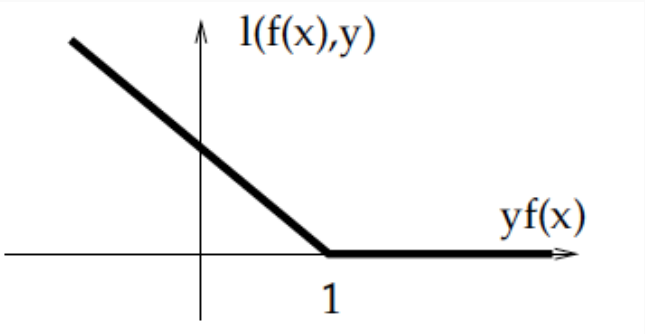
It is the kernelized version of the famous iteratively reweighted least-square (IRLS) method to solve the standard linear logistic regression.

SVM

- Historically the first "kernel method" for pattern recognition, still the most popular.
- Often state-of-the-art in performance.
- One particular choice of loss function (hinge loss).
- Leads to a sparse solution, i.e., not all points are involved in the decomposition (compression).
- Particular algorithm for fast optimization (decomposition by chunking methods).

- Hinge loss:

$$\varphi_{hinge}(u) = \max(1 - u, 0)$$



- SVM solves:

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \varphi_{hinge}(y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2$$

- By the representer theorem, the solution satisfies

$$\hat{f}(x) = \sum_{i=1}^{n} \hat{a}_i K(x_i, x)$$

  where $\hat{a}$ solves

$$\min_{a \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^{n} \varphi_{hinge}(y_i [Ka]_i) + \lambda a^T K a$$

- This is a convex optimization problem
- But the objective function is not smooth (because of the hinge loss)

- Let us introduce additional slack variables $\xi_1, \ldots, \xi_n \in \mathbb{R}$. The problem is equivalent to:

$$\min_{a \in \mathbb{R}^n, \xi \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^{n} \xi_i + \lambda a^T K a$$

  subject to

$$\xi_i \geq \varphi_{hinge}(y_i [Ka]_i)$$

- The objective function is now smooth, but not the constraints
- However it is easy to replace the non-smooth constraint by a conjunction of two smooth constraints, because:

$$u \geq \varphi_{hinge}(v) \Leftrightarrow u \geq 1 - v, v \geq 0$$

SVM primal formulation

$$\hat{f}(x) = \sum_{i=1}^{n} \hat{a}_i K(x_i, x)$$

where $\hat{a}$ solves

$$\min_{a \in \mathbb{R}^n, \xi \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^{n} \xi_i + \lambda a^T K a$$

subject to

$$y_i [Ka]_i + \xi_i - 1 \geq 0, \quad \xi_i \geq 0, i = 1, \quad \ldots, n$$

This is a classical quadratic program (minimization of a convex quadratic function with linear constraints) for which any out-of-the-box optimization package can be used.

The dimension of the problem and the number of constraints, however, are $2n$ where n is the number of points. General-purpose QP solvers will have difficulties when n exceeds a few thousands.

Solving the dual of this problem (also a QP) will be more convenient and lead to faster algorithms (due to the sparsity of the final solution).

Lagrangian

- Let us introduce the Lagrange multiplier $\mu, \nu \in \mathbb{R}^n$
- The Lagrangian of the problem is

$$L(a, \xi, \mu, \nu) = \frac{1}{n}\xi_i + \lambda a^T K a - \sum_{i=1}^{n} \mu_i [y_i [Ka]_i + \xi_i - 1] - \sum_{i=1}^{n} \nu_i \xi_i$$

or in matrix notations

$$L(a, \xi, \mu, \nu) = \frac{1}{n}\xi^T 1 + \lambda a^T K a - (\text{diag}(y)\mu)^T K a - (\mu + \nu)^T \xi + \mu^T 1$$

Minimize $L(a, \xi, \mu, \nu)$ w.r.t. a:

$$\nabla_a L = 2\lambda K a - K \text{diag}(y)\mu = K(2\lambda a - \text{diag}(y)\mu)$$

$$a^* = \frac{\text{diag}(y)\mu}{2\lambda}$$

Minimize $L(a, \xi, \mu, \nu)$ w.r.t. ξ:

$L(a, \xi, \mu, \nu)$ is a linear function in ξ

Its minimum is $-\infty$ except when it is constant, i.e., when

$$\nabla_\xi L = \frac{1}{n} - \mu - \nu = 0$$

$$\mu + \nu = \frac{1}{n}$$

Dual function

We obtain the Lagrange dual function

$$q(\mu,\nu) = \inf_{a\in\mathbb{R}^n,\xi\in\mathbb{R}^n} L(a,\xi,\mu,\nu)$$
$$= \begin{cases} \mu^T 1 - \frac{1}{4\lambda}\mu^T \mathrm{diag}(y) K \mathrm{diag}(y)\mu & \mu+\nu = \frac{1}{n} \\ -\infty & o.w. \end{cases}$$

The dual problem is

$$\max q(\mu,\nu), \quad \text{subject to} \quad \mu \geq 0, \nu \geq 0$$

Dual problem

- If $\mu_i > 1/n$ for some i, then there is no $\nu_i \geq 0$ such that $\mu_i + \nu_i = 1/n$, hence $q(\mu, \nu) = -\infty$.
- If $0 \leq \mu_i \leq 1/n$ for all i, then the dual function takes finite values that depend only on μ by taking $\nu_i = 1/n - \mu_i$.
- The dual problem is equvalent to

$$\max_{0 \leq \mu \leq 1/n} \mu^T 1 - \frac{1}{4\lambda} \mu^T \mathrm{diag}(y) K \mathrm{diag}(y) \mu$$

or

$$\max_{0 \leq \mu \leq 1/n} \sum_{i=1}^{n} \mu_i - \frac{1}{4\lambda} \sum_{i,j=1}^{n} y_i y_j \mu_i \mu_j K(x_i, x_j)$$

Back to the primal

- Once the dual problem is solved in μ we get a solution of the primal problem by $a = \text{diag}(y)\mu/2\lambda$
- Because the link is so simple, we can therefore directly plug this into the dual problem to obtain the QP that a must solve
- SVM dual formulation:

$$\max_{a \in \mathbb{R}^n} 2 \sum_{i=1}^{n} a_i y_i - \sum_{i,j=1}^{n} a_i a_j K(x_i, x_j) = 2a^T y - a^T K a$$

subject to

$$0 \leq y_i a_i \leq \frac{1}{2\lambda n}, \quad i = 1, \ldots, n$$

Complimentary slackness conditions

- The complimentary slackness conditions are, for $i = 1, \ldots, n$,

$$\mu_i[y_i f(x_i) + \xi_i - 1] = 0, \qquad \nu_i \xi_i = 0$$

- In terms of a, this can be rewritten as

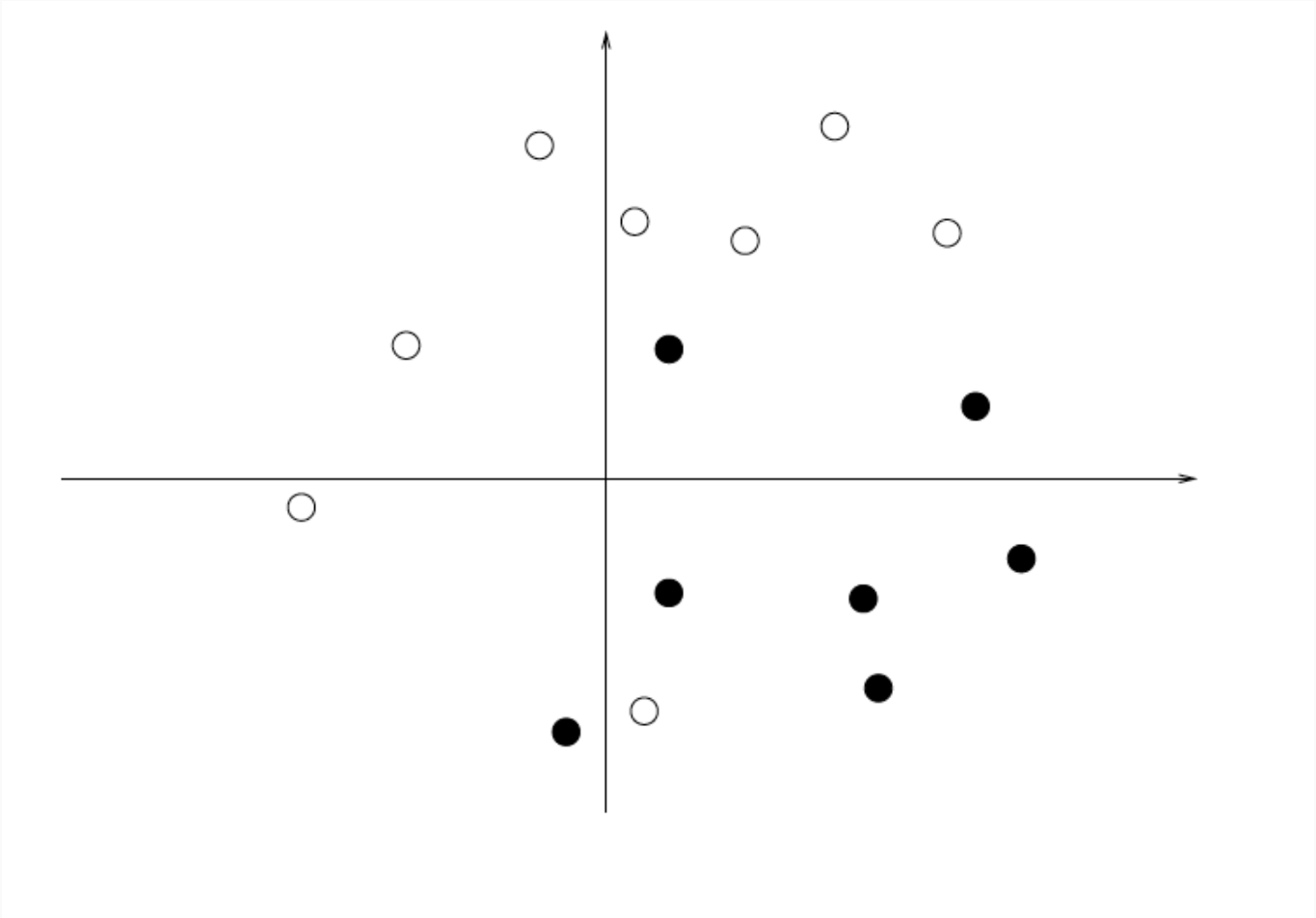
$$a_i[y_i f(x_i) + \xi_i - 1] = 0, \qquad (a_i - \frac{y_i}{2\lambda n})\xi_i = 0$$

Analysis of KKT conditions

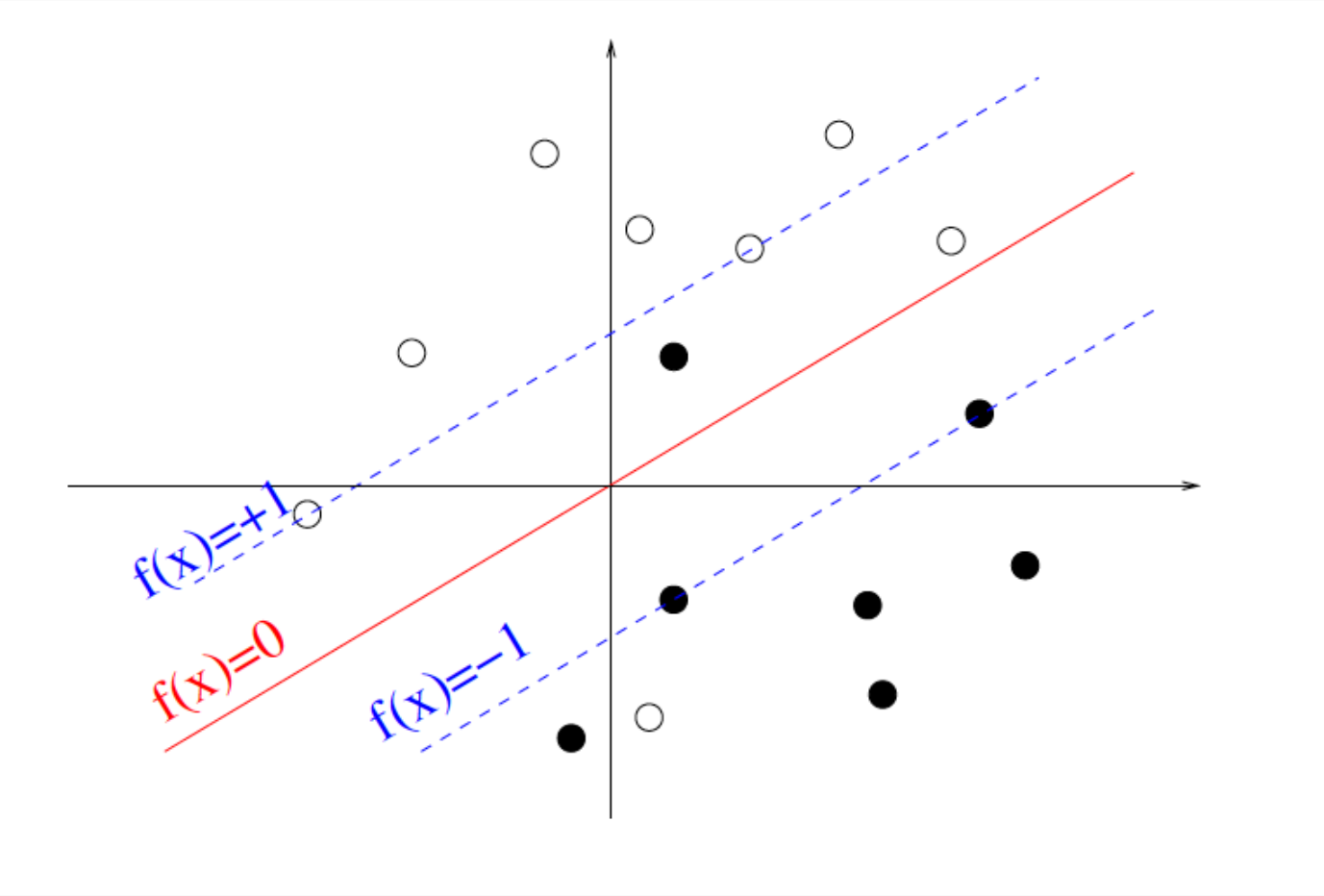
$$a_i[y_i f(x_i) + \xi_i - 1] = 0, \qquad (a_i - \frac{y_i}{2\lambda n})\xi_i = 0$$

- If $\alpha_i = 0$, then the second constraint is active, $\xi_i = 0$. This implies $y_i f(x_i) \geq 1$
- If $0 < y_i a_i < 1/(2\lambda n)$, the both constraints are active: $\xi_i = 0$ and $y_i f(x_i) + \xi_1 - 1 = 0$. This implies $y_i f(x_i) = 1$
- If $a_i = \frac{y_i}{2\lambda n}$, then the second constraint is not active ($\xi_i \geq 0$) while the first one is active: $y_i f(x_i) + \xi_i = 1$. This implies $y_i f(x_i) \leq 1$

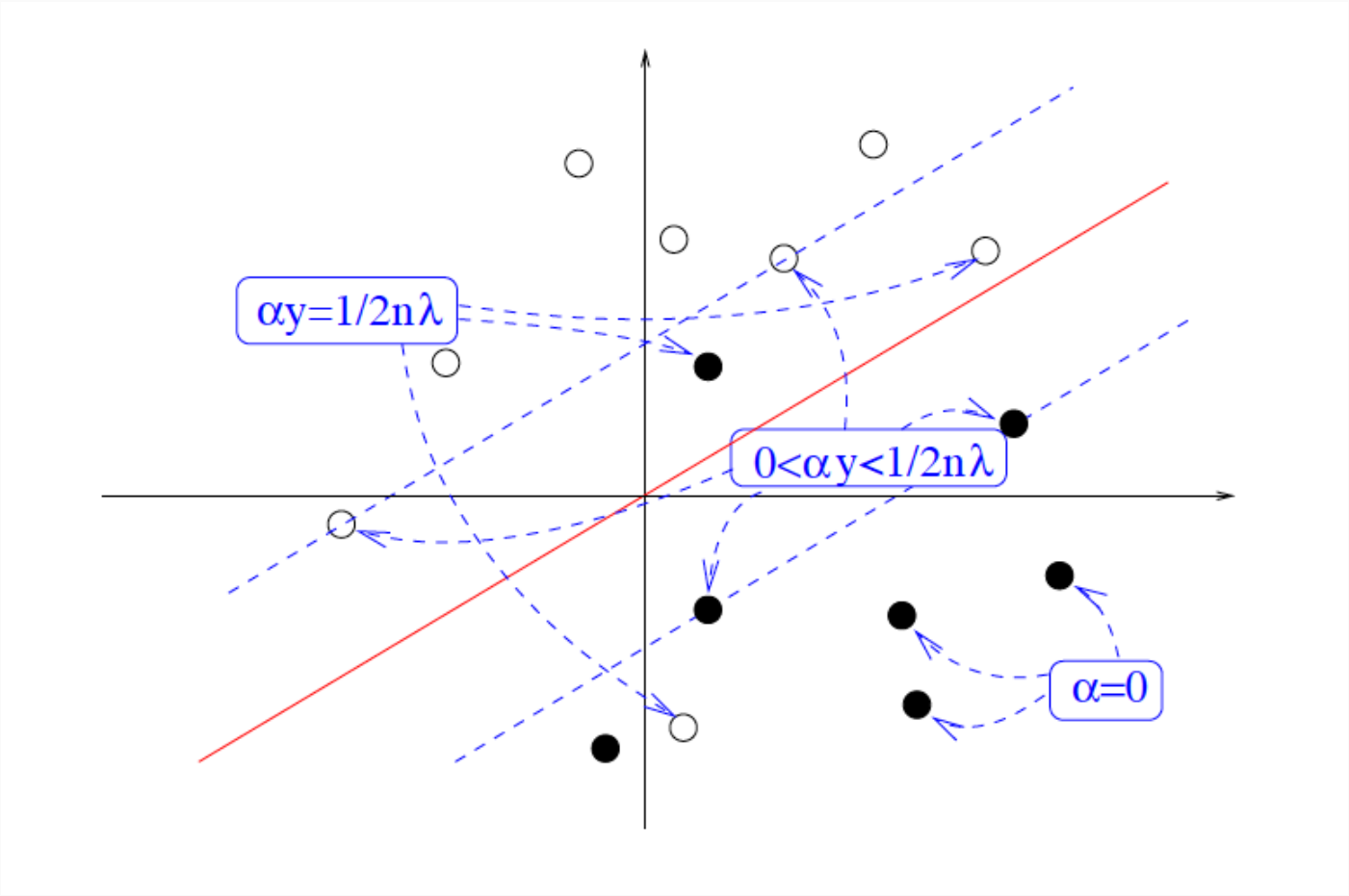
Geometric interpretation



Geometric interpretation



Geometric interpretation



Support vectors

Consequence of KKT conditions:

- The training points with $a_i \neq 0$ are called support vectors
- Only support vectors are important for the classification of new points

$$\forall x \in \mathcal{X}, f(x) = \sum_{i=1}^{n} a_i K(x_i, x) = \sum_{i \in SV} a_i K(x_i, x),$$

  where SV is the set of support vectors

Consequences:

- The solution is sparse in a, leading to fast algorithms for training (use of decomposition methods).
- The classification of a new point only involves kernel evaluations with support vectors (fast).

References

Machine Learning with Kernel Methods by Julien Mairal & Jean-Philippe Vert